



# Hidden Markov Anomaly Detection: From OC-SVM to Structured Sequence Models

Introduction to Machine Learning

Spring 1405

**Deadline:** 5 Mordad 1405

# Contents

|          |                                                                                  |           |
|----------|----------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Project Overview</b>                                                          | <b>3</b>  |
| 1.1      | Motivation . . . . .                                                             | 3         |
| <b>2</b> | <b>Introduction to Anomaly Detection</b>                                         | <b>3</b>  |
| 2.1      | Informal Definition . . . . .                                                    | 4         |
| 2.2      | Formal Problem Statement . . . . .                                               | 4         |
| 2.3      | Density Level Set View . . . . .                                                 | 5         |
| 2.4      | Connection to This Project . . . . .                                             | 6         |
| <b>3</b> | <b>OC-SVM Baseline</b>                                                           | <b>6</b>  |
| 3.1      | Why OC-SVM? . . . . .                                                            | 6         |
| 3.2      | Model and Objective . . . . .                                                    | 6         |
| 3.3      | Interpretation of $\nu$ . . . . .                                                | 8         |
| 3.4      | Implementation Requirements . . . . .                                            | 8         |
| <b>4</b> | <b>Latent Dependency and Real-World Motivation</b>                               | <b>10</b> |
| 4.1      | Why the i.i.d. Assumption Can Fail . . . . .                                     | 10        |
| 4.2      | Markov Chains . . . . .                                                          | 11        |
| 4.3      | Hidden Markov Models . . . . .                                                   | 12        |
| 4.4      | Most Likely State Sequence and Viterbi Decoding . . . . .                        | 13        |
| 4.5      | Connection to HMAAD . . . . .                                                    | 14        |
| 4.6      | Bonus Questions* . . . . .                                                       | 15        |
| <b>5</b> | <b>Hidden Markov Anomaly Detection</b>                                           | <b>15</b> |
| 5.1      | Conceptual Bridge from OC-SVM to HMAAD . . . . .                                 | 16        |
| 5.2      | Hidden Markov Joint Feature Map . . . . .                                        | 17        |
| 5.3      | Simplified HMAAD Training Algorithm . . . . .                                    | 18        |
| 5.4      | Prediction . . . . .                                                             | 19        |
| <b>6</b> | <b>Implementation Plan and Experiments</b>                                       | <b>20</b> |
| 6.1      | Dataset Candidates . . . . .                                                     | 20        |
| 6.1.1    | Candidate A: Synthetic Gaussian Sequence Data (Required) . . . . .               | 20        |
| 6.1.2    | Candidate B: Bioinformatics-Inspired Sequence Data (Bonus)* . . . . .            | 21        |
| 6.1.3    | Candidate C: Computational Sustainability / Wind Turbine Data (Bonus)* . . . . . | 21        |
| 6.2      | Recommended Final Dataset Choice . . . . .                                       | 21        |
| 6.2.1    | Data Generation and Access . . . . .                                             | 21        |
| 6.2.2    | Required Preprocessing Steps . . . . .                                           | 22        |
| 6.2.3    | Expected Runtime and Computational Scope . . . . .                               | 22        |
| 6.3      | Experimental Protocol . . . . .                                                  | 22        |
| 6.3.1    | Data Splits and Dataset Configurations . . . . .                                 | 22        |
| 6.3.2    | Anomaly Fraction Sensitivity Testing . . . . .                                   | 23        |
| 6.3.3    | Required Evaluation Metrics . . . . .                                            | 23        |
| 6.3.4    | Required Plots and Figures . . . . .                                             | 23        |
| 6.4      | Baseline Feature Choices for OC-SVM . . . . .                                    | 23        |
| 6.4.1    | Representation 1: Flattened Sequence Features . . . . .                          | 24        |
| 6.4.2    | Representation 2: Global Summary Statistics . . . . .                            | 24        |
| 6.4.3    | Representation 3: Window-Based Summary Statistics . . . . .                      | 24        |

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| 6.5      | HMAD Implementation Choices . . . . .             | 24        |
| 6.5.1    | Hidden State and Emission Configuration . . . . . | 25        |
| 6.5.2    | Joint Feature Vector Dimensionality . . . . .     | 25        |
| <b>7</b> | <b>Conclusion</b>                                 | <b>25</b> |

# 1 Project Overview

## 1.1 Motivation

Many real-world machine learning systems must detect unusual behavior without having a complete list of all possible failures, attacks, or abnormal patterns. For example, a wind turbine monitoring system may need to detect a short abnormal block in sensor readings, a computer security system may need to detect a suspicious sequence of events, and a bioinformatics system may need to identify unusual structure in a DNA sequence. In all of these cases, the main challenge is that anomalies are rare and may not be available as labeled training examples. A natural first approach is to learn what normal data look like and then reject examples that fall outside the learned normal region. This is the idea behind one-class anomaly detection. In this project, we begin with the one-class support vector machine (OC-SVM), which is one of the standard kernel-based methods for this setting. OC-SVM treats each input as a fixed-length feature vector and learns a boundary that contains most of the normal training data. However, many important data sources are not just unordered collections of independent points. They are sequences. In a sequence, the order of observations can be as important as the observations themselves. A single sensor value may look normal in isolation, but the transition into or out of that value may be physically impossible. A single event in a server log may be common, but a particular chain of events may indicate an attack. A single nucleotide in a DNA sequence may be ordinary, but the hidden region structure of the sequence may be unusual. This motivates the main question of the project:

*When does a sequence-aware anomaly detector improve over a fixed-vector anomaly detector such as OC-SVM?*

To answer this question, we will move step by step from OC-SVM to a simplified version of Hidden Markov Anomaly Detection (HMAD). The key idea is to represent each sequence not only by its observed values, but also by a hidden state path that explains how the sequence evolves. This hidden state path allows the detector to capture transition patterns and state-dependent behavior that ordinary summary features may miss. By the end of the project, students will have implemented both a classical OC-SVM baseline and a simplified HMAD-style model. They will compare these methods on controlled synthetic sequence data and analyze when modeling latent sequential structure helps anomaly detection.

## 2 Introduction to Anomaly Detection

Anomaly detection is the problem of identifying observations that do not follow the dominant or expected pattern of the data. In this project, we study anomaly detection in an unsupervised or one-class setting: during training, we assume that the available data are mostly normal, and we do not rely on a large labeled collection of anomalous examples.

This setting is different from ordinary binary classification. In binary classification, the learning algorithm usually receives examples from both classes, such as “normal” and “anomaly”, together with their labels. In anomaly detection, anomalous examples may be rare, expensive to label, unknown in advance, or even absent from the training set. Therefore, the central goal is not to learn a boundary between two fully labeled classes. Instead, the goal is to learn what the normal data look like and then flag examples that deviate from this learned notion of normality.

Examples of anomalies include:

- **Point anomaly:** a single sensor reading is far outside the usual range, such as an unusually large vibration value in a wind turbine.

- **Contextual anomaly:** a value may be normal in one context but abnormal in another, such as a high temperature that is acceptable during heavy load but suspicious during idle operation.
- **Collective or sequence anomaly:** individual observations may look normal by themselves, but their order or pattern is unusual. For example, a short block of shifted sensor readings, a suspicious sequence of network events, or a DNA segment with unusual hidden structure may indicate an anomaly.

The last case is especially important for this project. A fixed-vector anomaly detector such as OC-SVM can be a strong baseline, but it does not explicitly model hidden temporal or sequential structure. This motivates the later transition from OC-SVM to a simplified Hidden Markov Anomaly Detection (HMAD) framework.

## 2.1 Informal Definition

At an intuitive level, an anomaly is an observation that is rare or inconsistent with the majority of the data. If most data points are generated by a normal process, then an anomaly is a point or sequence that does not appear to be well explained by that process.

For example, suppose we monitor a machine using a temperature sensor. If most readings are around a stable operating range, then a single extremely large reading may be suspicious. However, not all anomalies are single extreme values. A sequence such as

$$60^{\circ}\text{C} \rightarrow 85^{\circ}\text{C} \rightarrow 60^{\circ}\text{C}$$

may also be suspicious if the machine cannot physically heat up and cool down that quickly, even if each individual temperature value is possible. This illustrates why some anomalies are structural: they are caused by the relationship between observations, not only by the value of one observation.

In this project, we will use this idea repeatedly. OC-SVM will first be used to learn a normal region in a fixed feature space. Later, HMAD-style models will use latent states to represent hidden sequence structure and detect anomalies that are difficult to capture with ordinary summary features.

**Theoretical Question 1.** Give one example of a point anomaly and one example of a sequence anomaly. Explain why the sequence anomaly may not be detectable by looking at each observation independently.

## 2.2 Formal Problem Statement

Let

$$x_1, x_2, \dots, x_n \in \mathcal{X}$$

be training examples. In the standard anomaly detection formulation, these examples are assumed to be generated mostly from an unknown nominal probability distribution  $P$ . In the cleanest version of the problem, all training examples are nominal. In more realistic settings, the training set may contain a small fraction of contaminated or anomalous samples.

The objective is to learn a scoring function

$$f : \mathcal{X} \rightarrow \mathbb{R},$$

where larger values of  $f(x)$  mean that  $x$  looks more normal, and smaller values mean that  $x$  looks more anomalous. A common decision rule is

$$\hat{y}(x) = \begin{cases} \text{normal}, & f(x) \geq 0, \\ \text{anomaly}, & f(x) < 0. \end{cases}$$

Equivalently, one may define an anomaly score

$$s(x) = -f(x),$$

so that larger values of  $s(x)$  correspond to more anomalous examples. This sign convention is important in the implementation part of the project, because some software libraries return normality scores rather than anomaly scores.

The learned detector also defines an estimated nominal set:

$$\hat{L} = \{x \in \mathcal{X} : f(x) \geq 0\}.$$

Inputs inside  $\hat{L}$  are accepted as normal, while inputs outside  $\hat{L}$  are flagged as anomalies.

**Theoretical Question 2.** Why is anomaly detection harder than ordinary binary classification when labeled anomalous examples are unavailable or unreliable?

### 2.3 Density Level Set View

A useful way to formalize anomaly detection is through density level sets. Suppose the nominal distribution has a density function  $p(x)$ . Normal examples are expected to lie in regions where  $p(x)$  is relatively large, while rare or atypical examples lie in low-density regions.

For a target anomaly fraction  $\nu \in (0, 1)$ , the ideal nominal region can be written as

$$L_\nu = \{x \in \mathcal{X} : p(x) \geq b_\nu\},$$

where the threshold  $b_\nu$  is chosen so that

$$P(X \notin L_\nu) = \nu.$$

Thus,  $L_\nu$  contains the most typical examples under the nominal distribution, and approximately a fraction  $\nu$  of the probability mass lies outside it. Points outside  $L_\nu$  are treated as anomalies.

In practice, we do not know the true density  $p(x)$ , so we cannot compute  $L_\nu$  directly. Instead, we learn an approximation

$$\hat{L}_\nu = \{x \in \mathcal{X} : f(x) \geq 0\}$$

from data. OC-SVM is one important method for learning such a region without explicitly estimating the full density. Later in the project, HMAD will extend this idea to sequence data by replacing ordinary fixed-vector features with features that depend on hidden state structure.

The parameter  $\nu$  has an important interpretation throughout the project. Informally, it controls how much data the model is willing to treat as outside the nominal region. A small  $\nu$  usually leads to a larger accepted region, while a larger  $\nu$  allows the model to reject more examples as potential anomalies.

**Theoretical Question 3.** Suppose  $\nu = 0.05$ . What does this say about the amount of probability mass that the ideal detector is allowed to place outside the nominal region?

## 2.4 Connection to This Project

This section gives the common language for the rest of the project. Section 3 introduces OC-SVM as a kernel-based method for learning an estimated nominal region  $\hat{L}_\nu$ . Section 4 then explains why the usual independent-data assumption can fail for sequences. Finally, Section 5 introduces a simplified HMAD-style method that uses hidden states to construct sequence-aware features before applying the one-class detection idea.

## 3 OC-SVM Baseline

**Section contact:** Farzan Aghaei    **Telegram:** @farzan\_aghaei

### 3.1 Why OC-SVM?

In the previous section, anomaly detection was introduced as the task of learning what “normal” data look like and assigning low normality scores to unusual observations. In this section, we study one of the most widely used baseline methods for this task: the one-class support vector machine (OC-SVM).

OC-SVM is useful when the training data are mostly nominal and reliable labeled examples of anomalies are not available. This is different from ordinary binary classification, where both normal and anomalous examples are usually provided with labels. Instead of learning a boundary between two labeled classes, OC-SVM learns a region that contains most of the nominal training examples. Test examples that fall outside this region are treated as potential anomalies.

OC-SVM is a particularly important baseline for this project for two reasons. First, it is a standard kernel-based anomaly detection method and is therefore a reasonable method to compare against. Second, OC-SVM treats each input as a fixed-length feature vector. This makes it simple and effective in many settings, but it also means that it does not explicitly model temporal order, hidden states, or sequential dependency. Later in the project, this limitation will motivate a more sequence-aware anomaly detection method.

**Theoretical Question 4.** Why is OC-SVM called a “one-class” method? Explain how its training setting differs from ordinary binary classification.

**Theoretical Question 5.** Why is OC-SVM a meaningful baseline for this project, even though the project later studies sequence-aware anomaly detection?

### 3.2 Model and Objective

Let

$$x_1, \dots, x_n \in \mathcal{X}$$

be training examples that mostly come from the nominal data-generating process. OC-SVM maps each input  $x$  into a feature space using a feature map

$$\phi : \mathcal{X} \rightarrow \mathcal{H}.$$

In this feature space, the method tries to separate most of the mapped training examples from the origin by a hyperplane.

The OC-SVM scoring function is

$$f_{w,\rho}(x) = \langle w, \phi(x) \rangle - \rho,$$

where  $w$  is the normal vector of the hyperplane and  $\rho$  is an offset. The score  $f_{w,\rho}(x)$  is a normality score: larger values indicate that  $x$  looks more normal according to the learned model, while smaller values indicate that  $x$  is more suspicious. A common decision rule is

$$\hat{y}(x) = \begin{cases} \text{normal}, & f_{w,\rho}(x) \geq 0, \\ \text{anomaly}, & f_{w,\rho}(x) < 0. \end{cases}$$

A standard primal formulation of OC-SVM is

$$\begin{aligned} \min_{w,\rho,\xi} \quad & \frac{1}{2}\|w\|^2 - \rho + \frac{1}{\nu n} \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & \langle w, \phi(x_i) \rangle \geq \rho - \xi_i, \quad i = 1, \dots, n, \\ & \xi_i \geq 0. \end{aligned}$$

The terms in this objective have the following interpretation:

- $\frac{1}{2}\|w\|^2$  is a regularization term. It discourages overly complex decision boundaries.
- $-\rho$  encourages the hyperplane to move away from the origin so that a large part of the nominal data is accepted as normal.
- $\xi_i$  are slack variables. They allow some training examples to violate the boundary.
- $\nu$  controls the trade-off between including most training examples and allowing some examples to be treated as outliers.

The feature map  $\phi(x)$  does not need to be computed explicitly. Instead, OC-SVM can use a kernel function

$$k(x, x') = \langle \phi(x), \phi(x') \rangle.$$

This is useful because it allows the method to learn nonlinear boundaries in the original input space. A common choice is the radial basis function (RBF) kernel:

$$k(x, x') = \exp(-\gamma\|x - x'\|^2),$$

where  $\gamma$  controls the locality of the kernel. A very small  $\gamma$  usually produces a smoother boundary, while a very large  $\gamma$  may produce a boundary that is too sensitive to individual training examples.

**Theoretical Question 6.** What is the geometric meaning of separating the mapped training data from the origin in the OC-SVM feature space?

**Theoretical Question 7.** Why are slack variables  $\xi_i$  needed in the OC-SVM objective? Relate your answer to noisy or contaminated training data.

### 3.3 Interpretation of $\nu$

The parameter  $\nu \in (0, 1]$  is one of the main hyperparameters of OC-SVM. It controls how much of the training data the model is allowed to treat as outside the nominal region.

Intuitively:

- A smaller  $\nu$  makes the model less willing to reject training examples. This usually leads to a larger accepted region.
- A larger  $\nu$  allows the model to reject more training examples. This usually leads to a tighter accepted region.

In the standard theoretical interpretation,  $\nu$  is related to the expected fraction of outliers and support vectors. More precisely, under standard assumptions, it acts as an upper bound on the fraction of training outliers and a lower bound on the fraction of support vectors. In this project, you are not required to prove this property. You should understand  $\nu$  as a practical parameter connected to the expected contamination level in the training data.

For example, if you believe that approximately 10% of the training examples may be anomalous or corrupted, then  $\nu = 0.1$  is a reasonable initial value. However,  $\nu$  is still a hyperparameter, so it is often useful to compare several values, such as

$$\nu \in \{0.01, 0.05, 0.1, 0.2\}.$$

**Theoretical Question 8.** Explain the role of  $\nu$  in OC-SVM. What may happen if  $\nu$  is chosen much smaller or much larger than the true contamination level?

### 3.4 Implementation Requirements

In this project, you should implement OC-SVM using `scikit-learn`. You are not required to implement the quadratic optimization problem from scratch. However, your report must clearly explain the model objective, the role of  $\nu$ , the kernel choice, the preprocessing steps, and the evaluation metric.

OC-SVM expects each input to be represented as a fixed-length vector. Therefore, if the dataset contains sequences, each sequence must first be converted into a vector representation. This conversion is part of the baseline design. Possible choices include:

- Flattened sequence features:** concatenate all time steps into one vector. This preserves position information when all sequences have the same length.
- Summary statistics:** compute features such as mean, standard deviation, minimum, maximum, and median. This representation is compact but may lose temporal information.
- Window-based summaries:** divide each sequence into fixed windows and compute summary statistics within each window. This keeps some local temporal information while still producing fixed-length vectors.

This feature-construction step is important. A sequence anomaly may not always appear as a single unusual value. Sometimes the anomaly is caused by the order, duration, or arrangement of observations. In such cases, simple fixed-vector summaries may miss important structure. This is one reason why the later parts of the project consider sequence-aware models.

**Theoretical Question 9.** Why does converting a sequence into a fixed-length vector not necessarily mean that temporal dependency has been properly modeled?

A recommended OC-SVM implementation interface is shown below. The scaler should be fitted only on the training data and then applied to the test data.

Listing 1: Suggested OC-SVM baseline interface.

```
from sklearn.preprocessing import StandardScaler
from sklearn.svm import OneClassSVM
from sklearn.metrics import roc_auc_score

# X_train: training features, mostly nominal examples
# X_test: test features
# y_test: test labels, where 0 = normal and 1 = anomaly

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = OneClassSVM(kernel="rbf", gamma="scale", nu=0.1)
model.fit(X_train_scaled)

scores = model.decision_function(X_test_scaled)
pred = model.predict(X_test_scaled) # +1 normal, -1 anomaly

# decision_function returns normality scores.
# If y_test = 1 means anomaly, use negative scores as anomaly scores.
anomaly_scores = -scores
auc = roc_auc_score(y_test, anomaly_scores)

print("ROC-AUC:", auc)
```

The output of `decision_function` should be interpreted carefully. Larger values mean that the example is more normal according to the model. Therefore, if anomalies are encoded as label 1, then the anomaly score should be `-scores` when computing ROC-AUC.

You must report:

- the sequence feature representation used for OC-SVM;
- preprocessing steps, especially standardization;
- the kernel type;
- the value of  $\nu$ ;
- kernel parameters such as `gamma`;
- ROC-AUC on the test set;
- optionally, precision, recall, and F1-score at a chosen threshold.

**Simulation Question 1.** Implement an OC-SVM baseline using `scikit-learn` on the selected dataset. Report the feature representation, preprocessing, kernel,  $\nu$ , kernel parameters, and ROC-AUC.

**Simulation Question 2.** Compare at least two sequence feature representations for OC-SVM, such as flattened features and summary or window-based features. Which representation works better, and why?

**Simulation Question 3.** Evaluate OC-SVM for several values of  $\nu$ , for example  $\{0.01, 0.05, 0.1, 0.2\}$ . Report how ROC-AUC changes and briefly interpret the result.

#### Student Deliverable

Submit the OC-SVM baseline results as part of your report. Your report should clearly state the feature representation, preprocessing steps, kernel,  $\nu$ , kernel parameters, and ROC-AUC. You should also briefly discuss one limitation of OC-SVM for sequence anomaly detection and explain why this limitation motivates the sequence-aware method introduced in the next parts of the project.

## 4 Latent Dependency and Real-World Motivation

**Section contact:** Bahador Hadi    **Telegram:** @Bahador\_Hadi

The OC-SVM baseline in Section 3 treats each input as a fixed-length feature vector. This is a reasonable starting point, but it does not explicitly model how observations are ordered inside a sequence. In many real-world anomaly detection problems, this ordering is crucial. A single observation may look normal by itself, while the sequence of observations may be highly unusual.

For example, consider a machine sensor, a server event log, or a biological sequence. In each case, the meaning of the current observation depends partly on what came before it. A temperature value may be acceptable during heavy load but suspicious immediately after an idle period. A server login may be normal by itself but suspicious if it occurs immediately after a port scan. A nucleotide in a DNA sequence may be common by itself, while the surrounding hidden region structure may be unusual.

This section introduces the sequence-modeling ideas needed for the simplified HMA method: latent dependency, Markov chains, hidden Markov models, and Viterbi decoding.

### 4.1 Why the i.i.d. Assumption Can Fail

Many classical machine learning methods assume that training examples are independent and identically distributed, often abbreviated as i.i.d. If a sequence

$$X = (X_1, X_2, \dots, X_T)$$

is treated as i.i.d., then its joint probability is modeled as

$$P(X_1 = x_1, \dots, X_T = x_T) = \prod_{t=1}^T P(X_t = x_t).$$

This model only asks whether each individual observation is likely. It does not ask whether the transition from one observation to the next is likely.

This can be problematic for anomaly detection. Some anomalies are not caused by a single impossible value, but by an unusual pattern over time. These are often called collective anomalies or sequence anomalies. In such cases, a detector that ignores ordering may fail.

**Example: machine sensor sequence.** Suppose a wind turbine gearbox temperature is usually around  $60^{\circ}\text{C}$ , but values near  $85^{\circ}\text{C}$  are still possible during high load. A static model may not flag  $85^{\circ}\text{C}$  as anomalous because the value itself is not impossible. However, the short sequence

$$60^{\circ}\text{C} \rightarrow 85^{\circ}\text{C} \rightarrow 60^{\circ}\text{C}$$

may be physically suspicious if the machine cannot heat up and cool down that quickly. The anomaly is therefore in the transition pattern, not only in the individual values.

**Example: server event logs.** Consider server events

$$s_1 = \text{Port Scan}, \quad s_2 = \text{Login}, \quad s_3 = \text{Data Download}.$$

Each event may occur normally in isolation. However, the sequence

$$\text{Port Scan} \rightarrow \text{Login} \rightarrow \text{Data Download}$$

may be suspicious under a nominal system model because this specific ordering may have low probability during normal behavior.

**Theoretical Question 10.** Explain why a detector that only looks at individual observations may fail to detect a sequence anomaly. Give one example where each individual observation is normal, but the ordering is suspicious.

**Theoretical Question 11.** For the server-log example above, explain why the sequence

$$\text{Port Scan} \rightarrow \text{Login} \rightarrow \text{Data Download}$$

could have low probability under a nominal transition model, even if each event has high marginal probability.

## 4.2 Markov Chains

A Markov chain is a simple probabilistic model for sequences of states. Let

$$z_1, z_2, \dots, z_T$$

be a sequence of states, where each state belongs to a finite state space

$$\mathcal{Y} = \{1, 2, \dots, K\}.$$

The first-order Markov assumption states that the next state depends only on the current state:

$$P(z_t \mid z_1, \dots, z_{t-1}) = P(z_t \mid z_{t-1}).$$

This assumption is useful because it captures local dependency while keeping the model simple.

A time-homogeneous Markov chain is specified by:

- an initial state distribution

$$\pi_i = P(z_1 = i), \quad i = 1, \dots, K,$$

where  $\sum_{i=1}^K \pi_i = 1$ ;

- a transition matrix

$$A_{ij} = P(z_t = j \mid z_{t-1} = i), \quad i, j \in \{1, \dots, K\}.$$

Each row of  $A$  is a probability distribution:

$$\sum_{j=1}^K A_{ij} = 1, \quad A_{ij} \geq 0.$$

Under this model, the probability of a complete state sequence  $z = (z_1, \dots, z_T)$  is

$$P(z \mid \pi, A) = \pi_{z_1} \prod_{t=2}^T A_{z_{t-1}, z_t}.$$

**Theoretical Question 12.** Consider a two-state Markov chain with states 1 = normal and 2 = high-load. What does a large value of  $A_{11}$  mean? What does a large value of  $A_{12}$  mean?

**Simulation Question 4.** Write a Python function that computes the log-probability of a state sequence  $z = (z_1, \dots, z_T)$  under a given initial distribution  $\pi$  and transition matrix  $A$ :

$$\log P(z \mid \pi, A) = \log \pi_{z_1} + \sum_{t=2}^T \log A_{z_{t-1}, z_t}.$$

Use log-probabilities rather than raw products to avoid numerical underflow.

### 4.3 Hidden Markov Models

In a Markov chain, the states  $z_t$  are directly observed. In many real applications, however, the true state of the system is hidden. We only observe measurements generated from those hidden states. This leads to a hidden Markov model, or HMM.

An HMM contains two sequences:

- a hidden state sequence

$$z = (z_1, \dots, z_T),$$

where  $z_t \in \mathcal{Y} = \{1, \dots, K\}$ ;

- an observed sequence

$$x = (x_1, \dots, x_T).$$

The hidden states evolve according to a Markov chain, and each observation  $x_t$  is generated from the current hidden state  $z_t$ . The standard HMM factorization is

$$P(x, z) = P(z_1)P(x_1 \mid z_1) \prod_{t=2}^T P(z_t \mid z_{t-1})P(x_t \mid z_t).$$

The model parameters are usually written as

$$\lambda = (\pi, A, B),$$

where  $\pi$  is the initial state distribution,  $A$  is the transition matrix, and  $B$  describes the emission model.

For discrete observations,  $B$  can be an emission probability matrix:

$$B_{ik} = P(x_t = v_k \mid z_t = i).$$

For continuous observations, which we use in the implementation part of this project, the emission probability is replaced by an emission density. For example, with Gaussian emissions,

$$P(x_t \mid z_t = i) = \mathcal{N}(x_t; \mu_i, \sigma_i^2).$$

Thus, each hidden state has its own mean and variance.

**Interpretation.** The hidden state sequence provides a structural explanation of the observations. In a machine monitoring example, hidden states might correspond to normal operation and high-load operation. In a biological sequence, hidden states might correspond to different functional regions. In a server log, hidden states might correspond to different modes of system behavior.

**Theoretical Question 13.** What is the difference between a Markov chain and a hidden Markov model? In your answer, distinguish between hidden states  $z_t$  and observations  $x_t$ .

**Theoretical Question 14.** In the Gaussian-emission HMM used in this project, each hidden state  $i$  has parameters  $\mu_i$  and  $\sigma_i^2$ . What do these parameters represent?

**Simulation Question 5.** Implement a function that generates a sequence from a two-state Gaussian-emission HMM. Your function should take  $\pi$ ,  $A$ , emission means  $(\mu_1, \mu_2)$ , emission variances  $(\sigma_1^2, \sigma_2^2)$ , and sequence length  $T$ . It should return both the hidden state sequence  $z$  and the observed sequence  $x$ .

## 4.4 Most Likely State Sequence and Viterbi Decoding

In an HMM, the hidden states are not observed. Given only the observation sequence

$$x = (x_1, \dots, x_T),$$

we often want to infer the most likely hidden state sequence:

$$z^* = \arg \max_{z \in \mathcal{Z}} P(x, z \mid \lambda).$$

A naive approach would enumerate all possible hidden state sequences. If there are  $K$  hidden states and the sequence length is  $T$ , then there are  $K^T$  possible state paths. This is computationally infeasible even for moderate  $T$ .

The Viterbi algorithm solves this problem efficiently using dynamic programming. Instead of checking every complete path, it stores the best partial path ending in each state at each time step.

Define the Viterbi score

$$\delta_t(j) = \max_{z_1, \dots, z_{t-1}} P(z_1, \dots, z_{t-1}, z_t = j, x_1, \dots, x_t \mid \lambda).$$

For a discrete-emission HMM, the recursion is

$$\delta_{t+1}(j) = \left[ \max_{1 \leq i \leq K} \delta_t(i) A_{ij} \right] B_j(x_{t+1}).$$

For a Gaussian-emission HMM,  $B_j(x_{t+1})$  is replaced by the Gaussian density

$$\mathcal{N}(x_{t+1}; \mu_j, \sigma_j^2).$$

In practice, Viterbi is implemented in log-space:

$$\tilde{\delta}_{t+1}(j) = \max_{1 \leq i \leq K} [\tilde{\delta}_t(i) + \log A_{ij}] + \log P(x_{t+1} | z_{t+1} = j).$$

This avoids numerical underflow and converts products into sums.

To recover the best hidden path, the algorithm also stores backpointers indicating which previous state gave the maximum at each step. After the last time step, backtracking through these pointers gives the most likely hidden state sequence  $z^*$ .

**Theoretical Question 15.** Why is brute-force decoding of the most likely hidden state sequence infeasible for long sequences? What is the number of possible paths when there are  $K$  states and sequence length  $T$ ?

**Simulation Question 6.** Implement log-space Viterbi decoding for a two-state Gaussian-emission HMM. Test it on a sequence generated by your HMM generator. Report the fraction of time steps where the decoded hidden state matches the true generated hidden state.

## 4.5 Connection to HMAD

The goal of this section was not to study HMMs as a standalone topic, but to prepare the tools needed for Hidden Markov Anomaly Detection.

In the simplified HMAD method used in this project, the hidden state sequence  $z$  is used to construct a sequence-aware feature representation  $\Psi(x, z)$ . This representation contains information about both:

- how the hidden states transition over time;
- what observations are emitted while the sequence is in each hidden state.

The Viterbi algorithm is used to infer the most likely hidden state path for each observed sequence. Then the joint feature map  $\Psi(x, z)$  is passed to a one-class detector. In this way, HMAD keeps the one-class anomaly detection idea of OC-SVM, but gives it a feature representation that can capture hidden sequential structure.

**Theoretical Question 16.** In one or two paragraphs, explain why Viterbi decoding is useful for HMAD. What role does the decoded hidden state sequence play in constructing the feature vector  $\Psi(x, z)$ ?

### Student Deliverable

Students should demonstrate that they understand why sequence structure matters for anomaly detection. In particular, they should explain the difference between independent observations, Markov chains, and hidden Markov models; describe the role of hidden states and emission distributions; and explain why Viterbi decoding is needed to infer a likely hidden state path. For the implementation part, students should provide their Markov-chain log-probability function, their Gaussian-HMM sequence generator, and their log-space Viterbi decoder, together with a short discussion of how accurately Viterbi recovers the true hidden states on synthetic data.

## 4.6 Bonus Questions\*

The following questions are optional. They go beyond the minimum requirements of the project and are intended for students who want to explore more advanced aspects of sequence modeling.

**Theoretical Question 17.** A stationary distribution  $v$  of a Markov chain satisfies

$$v^\top A = v^\top, \quad \sum_{i=1}^K v_i = 1.$$

For the transition matrix

$$A = \begin{pmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{pmatrix},$$

compute the stationary distribution and interpret it as the long-run fraction of time spent in each state.

**Theoretical Question 18.** The entropy rate of a Markov chain with stationary distribution  $v$  and transition matrix  $A$  is

$$H = - \sum_{i=1}^K v_i \sum_{j=1}^K A_{ij} \log A_{ij}.$$

Explain qualitatively how a transition matrix with highly predictable transitions differs from a transition matrix with nearly uniform transitions. How might this be useful for anomaly detection?

**Theoretical Question 19.** For an HMM, the marginal likelihood of an observation sequence is

$$P(x \mid \lambda) = \sum_{z \in \mathcal{Z}} P(x, z \mid \lambda).$$

Explain why computing this sum by brute force is infeasible. Then state the main idea of the Forward algorithm and how it reduces the computational complexity.

**Theoretical Question 20.** Posterior decoding chooses the most likely state at each individual time step:

$$\hat{z}_t = \arg \max_i P(z_t = i \mid x, \lambda).$$

Viterbi decoding chooses the globally most likely state sequence. Explain why posterior decoding can sometimes produce an invalid or unlikely global path, while Viterbi respects the transition structure of the model.

**Simulation Question 7.** Compare ordinary probability multiplication and log-space computation for a long Markov chain sequence. Generate a sequence of length  $T = 5000$ , compute its probability both as a raw product and as a log-probability sum, and show when the raw product begins to suffer from numerical underflow.

## 5 Hidden Markov Anomaly Detection

**Section contact:** Ramtin Moslemi    **Telegram:** @Ra88Mt109I53N7

## 5.1 Conceptual Bridge from OC-SVM to HMAD

In Section 2, anomaly detection was framed as learning a scoring function  $f$  that assigns high values to nominal observations and low values to anomalies. Section 3 showed that OC-SVM solves this by finding a hyperplane that separates the mapped training data from the origin in a kernel feature space. The scoring function takes the form

$$f(x) = \langle w, \phi(x) \rangle - \rho,$$

where  $\phi(x)$  is a fixed feature representation of  $x$ .

This approach works well when each input  $x$  is a fixed-length vector and observations are roughly independent. However, as discussed in Section 3, when the data are sequences the feature  $\phi(x)$  must first compress the entire sequence into one vector. Any temporal structure that is not captured by this compression will be invisible to the detector.

**Hidden Markov Anomaly Detection (HMAD)** generalizes OC-SVM to sequences by replacing the fixed feature map  $\phi(x)$  with a *joint* feature map  $\Psi(x, z)$  that depends on both the observed sequence  $x$  and a hidden state sequence  $z = (z_1, \dots, z_T)$ . The scoring function becomes

$$f(x) = \max_{z \in \mathcal{Z}} \langle w, \Psi(x, z) \rangle - \rho.$$

The key difference from OC-SVM is the maximization over hidden state sequences. Rather than scoring  $x$  with a single fixed feature vector, HMAD searches over all possible hidden state explanations and selects the one that makes  $x$  look most nominal under the learned model. If no hidden state assignment can make  $x$  look nominal, the score will be low and the sequence will be flagged as anomalous.

Intuitively,  $\Psi(x, z)$  encodes *how* the sequence behaves, not just *what* values it contains. A nominal sequence should be well explained by some hidden state path that respects normal transition dynamics and normal emission patterns. An anomalous sequence either forces unusual transitions or produces observations that are inconsistent with any nominal hidden state.

**Theoretical Question 21.** OC-SVM uses a fixed feature map  $\phi(x)$ , while HMAD uses a joint feature map  $\Psi(x, z)$  with a maximization over hidden states. Explain in your own words what this maximization is doing. Why can it help identify anomalies that a fixed feature map would miss? Use an example to make your case.

**Theoretical Question 22.** Suppose two sequences  $x^{(1)}$  and  $x^{(2)}$  have identical summary statistics (same mean, variance, min, and max). Can HMAD still assign them different anomaly scores? Explain why or why not.

**Simulation Question 8.** To build intuition for why the joint feature map captures structure that summary statistics miss, construct the following toy experiment in Python.

- a) Generate two sequences of length  $T = 100$ , both consisting of the same 50 values drawn from  $\mathcal{N}(0, 1)$  and 50 values drawn from  $\mathcal{N}(4, 1)$ , but arranged differently: in  $x^{(1)}$  the two groups form two contiguous blocks (the first 50 steps from  $\mathcal{N}(0, 1)$ , the last 50 from  $\mathcal{N}(4, 1)$ ); in  $x^{(2)}$  the values are randomly shuffled.
- b) Verify that the two sequences have nearly identical global summary statistics (mean, variance, min, max). Plot both sequences side by side.
- c) Manually assign hidden states  $z$  using a simple threshold rule:  $z_t = 1$  if  $x_t < 2$ , else  $z_t = 2$ . Compute the joint feature vector  $\Psi(x, z)$  for both sequences using  $K = 2$  and  $\phi(x_t) = x_t$ .

- d) Compare the joint feature vectors of the two sequences. Which entries differ the most, and why?

## 5.2 Hidden Markov Joint Feature Map

For a sequence  $x = (x_1, \dots, x_T)$  with hidden states  $z = (z_1, \dots, z_T)$  taking values in  $\mathcal{Y} = \{1, \dots, K\}$ , the joint feature map  $\Psi(x, z)$  is built from two components that capture different aspects of the sequence structure.

**Transition features.** These count how many times each state transition  $i \rightarrow j$  occurs along the hidden path:

$$c_{ij}(z) = \sum_{t=2}^T \mathbf{1}\{z_{t-1} = i, z_t = j\}, \quad i, j \in \mathcal{Y}.$$

There are  $K^2$  such counts, one for each ordered pair of states. A nominal sequence is expected to have transition counts consistent with a stable, persistent hidden Markov structure. A sequence with unusually frequent or rare transitions will produce a feature vector that looks different from nominal training sequences.

**Emission features.** These accumulate the observed values  $\phi(x_t)$  separately for each hidden state:

$$e_i(x, z) = \sum_{t=1}^T \mathbf{1}\{z_t = i\} \phi(x_t), \quad i \in \mathcal{Y}.$$

Here  $\phi(x_t)$  is a local feature of the single observation  $x_t$  (for example,  $\phi(x_t) = x_t$  for scalar observations, or a bag-of-words vector for discrete symbols). The emission features capture what kinds of observations are associated with each hidden state. If a sequence emits observations that are atypical for any state, the emission features will look unusual compared to the training distribution.

Putting the two components together, the simplified joint feature map is

$$\Psi(x, z) = \left( \left( c_{ij}(z) \right)_{i,j \in \mathcal{Y}}, \left( e_i(x, z) \right)_{i \in \mathcal{Y}} \right) \in \mathbb{R}^{K^2 + K \cdot d},$$

where  $d$  is the dimension of the local feature  $\phi(x_t)$ . For scalar sequences ( $d = 1$ ) and  $K = 2$  hidden states this gives  $4 + 2 = 6$  features, which is very compact and easy to train an OC-SVM on.

**Theoretical Question 23.** For  $K = 2$  hidden states and scalar observations ( $d = 1$ ), write out all entries of  $\Psi(x, z)$  explicitly. How many entries does the feature vector have? What does each entry represent?

**Simulation Question 9.** Write a Python function `compute_joint_features(x, z, K)` that computes  $\Psi(x, z)$  for a scalar sequence  $x$  and a hard state assignment  $z$ , using  $\phi(x_t) = x_t$ .

- Your function should return a single NumPy vector of length  $K^2 + K$ . The first  $K^2$  entries should be the normalized transition counts  $c_{ij}(z)/(T - 1)$  (so they sum to approximately 1), and the last  $K$  entries should be the per-state mean emissions  $e_i(x, z)/(\text{count of state } i + \epsilon)$ .
- Generate 200 nominal sequences of length  $T = 100$  from  $\mathcal{N}(0, 1)$  and 50 anomalous sequences by inserting a block of 20 steps from  $\mathcal{N}(4, 1)$  at a random position. Use a simple threshold rule ( $z_t = 1$  if  $x_t < 2$ , else  $z_t = 2$ ) to assign states.

- c) Compute  $\Psi(x_i, z_i)$  for every sequence. Plot the resulting  $250 \times 6$  feature matrix as a heatmap, with normal and anomalous sequences grouped. Which entries of  $\Psi$  visually separate the two groups?

### 5.3 Simplified HMAD Training Algorithm

Training HMAD requires learning the weight vector  $w$  and offset  $\rho$  of the scoring function. This is challenging because the joint features  $\Psi(x_i, z_i)$  depend on the hidden states  $z_i$ , which are not observed. The simplified algorithm used in this project resolves this by alternating between two steps: (1) inferring the hidden state sequences given the current model, and (2) re-training the OC-SVM on the inferred joint features. This is a course-friendly approximation of the original HMAD optimization method.

The procedure is summarized in Algorithm 1.

---

#### Algorithm 1 Simplified HMAD Training Loop

---

**Require:** sequences  $x_1, \dots, x_n$ , number of hidden states  $K$ , OC-SVM parameter  $\nu$ , maximum iterations  $M$

- 1: **Initialize** HMM parameters  $(\pi, \mathbf{A}, \mathbf{B})$ , e.g. using  $K$ -means on pooled observations
  - 2: **for**  $m = 1, \dots, M$  **do**
  - 3:   **for** each training sequence  $x_i$  **do**
  - 4:     Infer hidden state sequence  $z_i^{(m)} \leftarrow \text{Viterbi}(x_i; \pi, \mathbf{A}, \mathbf{B})$
  - 5:     Compute joint features  $\psi_i \leftarrow \Psi(x_i, z_i^{(m)})$
  - 6:   **end for**
  - 7:   Update HMM parameters from the inferred state sequences  $z_1^{(m)}, \dots, z_n^{(m)}$
  - 8:   Train an OC-SVM on  $\{\psi_i\}_{i=1}^n$  using OC-SVM  $\nu$
  - 9:   **if**  $z_i^{(m)} = z_i^{(m-1)}$  for all  $i$  (or fewer than 1% of assignments change) **then**
  - 10:     **break** ▷ early stopping: hidden states have converged
  - 11:   **end if**
  - 12: **end for**
  - 13: **return** trained OC-SVM (weight  $w$ , offset  $\rho$ ) and final HMM parameters
- 

**Initialization.** A simple initialization strategy is to run  $K$ -means with  $K$  clusters on all individual time steps pooled from all training sequences. Each cluster center becomes the mean of one Gaussian emission, and the initial transition matrix can be set to favor self-transitions (e.g.  $A_{kk} = 0.9$ , equal off-diagonal).

**HMM parameter update.** After Viterbi decoding in each iteration, the emission parameters  $(\mu_k, \sigma_k^2)$  can be re-estimated by computing the mean and variance of all observations assigned to state  $k$  across all sequences. The transition matrix can be re-estimated by normalizing the empirical transition counts  $c_{ij}$ .

**Why the alternation works.** In each iteration, the Viterbi step finds the hidden state sequence that best explains the observations under the current model. The OC-SVM step then learns what joint features look nominal given those hidden state assignments. As the two steps reinforce each other across iterations, the model progressively learns a consistent notion of what a nominal sequence looks like both in terms of hidden state dynamics and observed values.

**Theoretical Question 24.** The training algorithm alternates between a Viterbi step and an OC-SVM step. What would happen if the algorithm were run for only one iteration (i.e.  $M = 1$ )? How does running more iterations potentially improve the learned model?

**Simulation Question 10.** Implement the simplified HMAD training loop (Algorithm 1) in Python with  $K = 2$  hidden states and Gaussian emissions.

- a) Initialize emission parameters using  $K$ -means (with  $K = 2$ ) fitted on all individual time steps pooled from the training sequences. Set the initial transition matrix to  $A_{kk} = 0.9$  and equal off-diagonal entries.
- b) Run the training loop for up to  $M = 10$  iterations. After each iteration, record: (i) the fraction of hidden state assignments that changed from the previous iteration, and (ii) the mean OC-SVM decision score on the training set.
- c) Plot both quantities against the iteration number. At what iteration does the hidden state assignment stabilize? Does the mean OC-SVM decision score become more stable across iterations?
- d) Print the learned emission means  $\mu_1, \mu_2$  and the transition matrix  $\mathbf{A}$  at convergence. Do they match the structure of the data-generating process?

## 5.4 Prediction

Once training is complete, scoring a new test sequence  $x$  requires two steps that mirror the training loop.

**Step 1: Hidden state inference.** Run the Viterbi algorithm with the learned HMM parameters to find the most likely hidden state sequence:

$$z^* = \arg \max_{z \in \mathcal{Z}} P(x, z \mid \pi, \mathbf{A}, \mathbf{B}).$$

This selects the hidden-state explanation that best fits the sequence structure under the learned nominal model.

**Step 2: Scoring.** Compute the joint features from the inferred hidden states and apply the learned OC-SVM:

$$f(x) = \langle w, \Psi(x, z^*) \rangle - \rho.$$

A positive score ( $f(x) \geq 0$ ) means the sequence looks nominal; a negative score ( $f(x) < 0$ ) flags the sequence as anomalous.

The complete prediction pipeline is:

$$x \xrightarrow{\text{Viterbi}} z^* \xrightarrow{\Psi} \Psi(x, z^*) \xrightarrow{\text{OC-SVM}} f(x).$$

Note that Viterbi decoding is required at test time, not just during training. This means that the computational cost of prediction scales with sequence length  $T$  and the number of hidden states  $K$  (specifically  $\mathcal{O}(K^2T)$ , the same complexity as in Section 4's Viterbi discussion). For the sequence lengths used in this project this is fast.

**Theoretical Question 25.** During prediction, why is it necessary to run Viterbi decoding on the test sequence rather than using, for example, a fixed or randomly assigned hidden state sequence? What would go wrong if you skipped the Viterbi step and simply used  $z = (1, 1, \dots, 1)$  for every test sequence?

**Simulation Question 11.** Using the trained HMA model from the previous simulation question, implement the full prediction pipeline and evaluate it on the held-out test set.

- a) For each test sequence  $x$ , run Viterbi decoding to obtain  $z^*$ , compute  $\Psi(x, z^*)$ , and evaluate the OC-SVM score  $f(x)$ .
- b) Report ROC-AUC on the test set and compare it to the OC-SVM baseline from Section 3.
- c) To verify that Viterbi matters, repeat the evaluation using a trivial state assignment  $z_t = 1$  for all  $t$  and all sequences (skipping the Viterbi step). How does the AUC change? Does this confirm or contradict your answer to the theoretical question above?
- d) Plot the Viterbi-decoded hidden state path  $z^*$  for one correctly classified normal sequence and one correctly classified anomalous sequence side by side. Does the decoded path visually align with the anomaly block?

#### Student Deliverable

Students should provide pseudocode for both the training loop and the prediction procedure in their report. The pseudocode should be self-contained: a reader should be able to implement the method from it without referring back to this section. Students do not need to derive the original HMA optimization problem or prove any theoretical properties of the algorithm.

## 6 Implementation Plan and Experiments

**Section contact:** Mahdi Mansouri    **Telegram:** @MahdiM04

### 6.1 Dataset Candidates

To evaluate the performance and robustness of both the baseline and sequence-aware anomaly detection models, we present three distinct dataset frameworks. To ensure a reliable and accessible experimental setup, this project avoids any reliance on unavailable or proprietary data channels. Instead, we prioritize data structures that are straightforward to programmatically package or generate within a clean environment.

#### 6.1.1 Candidate A: Synthetic Gaussian Sequence Data (Required)

This dataset serves as the mandatory foundation for your experimental evaluation. It is strongly recommended because its underlying parameters are completely controllable, transparent, and easy to reproduce.

- **Nominal Sequences:** Normal sequences of a discrete-time horizon  $T$  are generated such that observations at each time step are drawn independently and identically from a standard normal distribution,  $X_t \sim \mathcal{N}(0, 1)$ .

- **Anomalous Sequences:** Anomalous sequences are constructed by generating a baseline nominal sequence and inserting one or more contiguous temporal blocks where observations deviate, being drawn from a shifted mean distribution,  $X_t \sim \mathcal{N}(\mu, 1)$  where  $\mu \neq 0$ .
- **Experimental Variations:** In your evaluation, you will be required to systematically vary the anomaly block length, the total number of anomalous blocks within a sequence, and the overall anomaly contamination fraction across the dataset.

### 6.1.2 Candidate B: Bioinformatics-Inspired Sequence Data (Bonus)\*

This optional dataset framework connects naturally to the structural gene-finding validation experiments described in the original HMAF literature. It introduces sequence structure through categorical data patterns.

- **Sequence Structure:** The data consists of DNA-like sequences characterized by transitions through distinct hidden functional regions, specifically moving across intergenic zones, start codons, exons, and stop codons.
- **Simplified Generation:** To keep the project scope manageable for an introductory course, students will work directly with a simplified synthetic DNA generation process rather than undergoing complex real-world biological preprocessing pipelines.

### 6.1.3 Candidate C: Computational Sustainability / Wind Turbine Data (Bonus)\*

This optional candidate focuses on real-world time-series monitoring within an industrial IoT framework, illustrating how physical constraints shape sequential data patterns.

- **Data Framework:** The application utilizes public or simulated time-series sensor logs originating from wind turbine gearbox operations.
- **Nominal vs. Anomalous States:** Under normal operating conditions, nominal sequences correspond to stable, predictable turbine workloads. Anomalous sequences are characterized by containing a distinct block representing an abrupt environmental or operational shift, such as an unexpected wind speed change, a high-turbulence regime, or a simulated mechanical fault.
- **Preparation Note:** While highly engaging and practical, choosing this option requires more careful data preparation and formatting compared to the synthetic Gaussian framework.

## 6.2 Recommended Final Dataset Choice

For the Spring 1405 offering of this course, the **Synthetic Gaussian Sequence Data** is the strictly required core dataset for all baseline implementations and comparative evaluations. The **Bioinformatics-Inspired** and **Wind Turbine Gearbox** datasets are designated as optional bonus extensions for students seeking deeper exploration into categorical sequence structure or engineering applications.

### 6.2.1 Data Generation and Access

Rather than downloading large external files, students will programmatically generate the required synthetic data using a standardized helper script provided on the course portal:

- **Source Script:** Download `generate_data.py` [LINK](#).
- **Fixed Random Seed:** To ensure experimental reproducibility across reports, you must initialize the generator using a fixed random state parameter (`random_state=1405`).

### 6.2.2 Required Preprocessing Steps

Before passing any sequence or summarized vector to the models, you must apply the following pipeline:

1. **Train/Test Separation:** Generate independent sequence pools for training and testing. Ensure that the training pool contains exclusively nominal sequences, while the test pool includes the designated contamination fraction of anomalous sequences.
2. **Feature Standardization:** Utilize a `StandardScaler` to mean-center and scale your feature matrices to unit variance. Crucially, the scaler must be fitted *only* on the training data and subsequently applied to transform the test data to prevent data leakage.

### 6.2.3 Expected Runtime and Computational Scope

- **OC-SVM Baseline:** Feature extraction (summary/flattening) and fitting the baseline scikit-learn model over  $n = 200$  sequences should execute in under **2 seconds** on a standard laptop.
- **Simplified HMAD Loop:** Your iterative training loop (alternating between Viterbi decoding and OC-SVM retraining for  $M \leq 10$  iterations) is computationally bounded by  $\mathcal{O}(M \cdot n \cdot K^2 T)$ . For a state space of  $K = 2$  and sequence lengths of  $T = 100$ , the complete training pipeline is highly efficient and expected to converge in less than **30 seconds**.

## 6.3 Experimental Protocol

To rigorously evaluate the benefits of incorporating sequential structure into an anomaly detection framework, you will conduct a comparative analysis between the static baseline and the sequence-aware model. You must strictly adhere to the following setup parameters, metrics, and visualization guidelines to ensure your results are reproducible and statistically sound.

Students will evaluate and compare:

1. **One-Class SVM (OC-SVM Baseline):** Tested across multiple input representations (e.g., flattened vs. summary-based features as outlined in Section 3.4).
2. **Simplified HMAD:** Implemented with a hidden state space size of  $K = 2$ , alternating Viterbi sequence decoding with structural feature map optimization as defined in Section 5.3.

### 6.3.1 Data Splits and Dataset Configurations

- **Training Partition:** Generate a training pool consisting exclusively of  $n = 200$  nominal (clean) sequences of length  $T = 100$ . Both models must be trained solely on this clean data.
- **Test Partition:** Construct a separate, held-out test set consisting of 100 nominal sequences mixed with anomalous sequences according to the chosen test anomaly fraction  $r_{\text{test}}$ . Each anomalous sequence must contain a single block of length  $T_{\text{anomaly}} = 20$  drawn from the shifted mean distribution  $\mathcal{N}(4, 1)$ .

### 6.3.2 Anomaly Fraction Sensitivity Testing

You must evaluate the robustness of both algorithms under varying test-set anomaly fractions. Re-run your evaluation pipeline across the following values:

$$r_{\text{test}} \in \{0.01, 0.05, 0.1, 0.2\}.$$

Here,  $r_{\text{test}}$  denotes the fraction of anomalous sequences in the test set. For each setting, record the continuous normality scores and hard binary classifications to study how performance scales with contamination density.

### 6.3.3 Required Evaluation Metrics

Your report must include quantitative performance summaries using the following metrics:

- **Receiver Operating Characteristic - Area Under Curve (ROC-AUC):** Computed using continuous anomaly scores on the held-out test set. Ensure that continuous scores are properly inverted (i.e.,  $\text{anomaly\_score} = -\text{normality\_score}$ ) so that true anomalies correspond to higher values in your ROC calculation.
- **Thresholded Classification Metrics:** Choose a single standard operational threshold at the zero-score boundary ( $f(x) = 0$ ). Report the resulting **Precision**, **Recall**, and **F1-score** achieved by both models at this decision boundary.

### 6.3.4 Required Plots and Figures

Your final report submission must contain the following three figures, clearly labeled with axes and legends:

- a. **Sequence Visualization:** A side-by-side time-series line plot illustrating one example nominal sequence and one example anomalous sequence from the test set. The inserted anomalous block window must be visually distinct or highlighted.
- b. **ROC Curves:** A single plot displaying the ROC curves of both the OC-SVM baseline and the simplified HMA model on the test partition at a fixed test anomaly fraction of  $r_{\text{test}} = 0.1$ .
- c. **Contamination Sensitivity Profile:** A line chart plotting the final Test ROC-AUC (y-axis) against the test anomaly fraction  $r_{\text{test}}$  (x-axis) for both competing models.

## 6.4 Baseline Feature Choices for OC-SVM

Because the standard One-Class Support Vector Machine (OC-SVM) formulation models individual observations as static, fixed-length feature vectors ( $x_i \in \mathbb{R}^D$ ), it cannot directly process a raw sequence matrix of shape  $T \times d$ . To establish a robust baseline, you must implement a feature-construction layer that compresses or maps the temporal sequence into a vector representation.

As required by Simulation Question 2, you must implement and compare at least two distinct sequence feature representations from the choices detailed below.

### 6.4.1 Representation 1: Flattened Sequence Features

For fixed-length sequences where position-dependent mapping is informative, you will concatenate all sequential observations into a single high-dimensional vector.

- **Mathematical Formulation:** For a scalar sequence  $x = (x_1, x_2, \dots, x_T)$ , the flattened vector is given by:

$$\phi_{\text{flatten}}(x) = [x_1, x_2, \dots, x_T]^T \in \mathbb{R}^T$$

- **Characteristics:** This approach fully preserves absolute temporal positioning and alignment. However, it lacks translation invariance and scales poorly if the sequence length  $T$  grows excessively large.

### 6.4.2 Representation 2: Global Summary Statistics

This configuration condenses the overall distributional properties of the sequence into a compact set of static descriptors, discarding absolute temporal order.

- **Mathematical Formulation:** You will map each sequence  $x$  to a 5-dimensional summary feature vector:

$$\phi_{\text{summary}}(x) = [\mu, \sigma, \min, \max, \text{median}]^T \in \mathbb{R}^5$$

where  $\mu$  is the empirical mean,  $\sigma$  is the sample standard deviation, and the remaining elements represent the sequence bounds and center.

- **Characteristics:** This representation is highly efficient and invariant to sequence length or temporal shifts, but it completely discards the conditional transition context driving the data stream.

### 6.4.3 Representation 3: Window-Based Summary Statistics

To strike a balance between global compression and fine-grained localization, you will partition each sequence into fixed, non-overlapping localized temporal windows.

- **Mathematical Formulation:** Divide the horizon  $T$  into  $W$  uniform intervals of length  $L = T/W$ . Within each interval  $w \in \{1, \dots, W\}$ , extract local summary statistics (such as the window mean  $\mu_w$  and local standard deviation  $\sigma_w$ ). The final feature vector is the concatenation of these windowed descriptors:

$$\phi_{\text{window}}(x) = [\mu_1, \sigma_1, \mu_2, \sigma_2, \dots, \mu_W, \sigma_W]^T \in \mathbb{R}^{2W}$$

- **Characteristics:** This preserves rough local temporal structure and regional dynamics while keeping the underlying dimensionality controlled and fixed.

## 6.5 HMAAD Implementation Choices

To ensure the implementation remains computationally tractable and small enough for an introductory course, you will implement a structured, low-dimensional version of the Hidden Markov Anomaly Detection (HMAAD) framework rather than the full non-convex optimization setup. You must strictly adhere to the following model configurations:

### 6.5.1 Hidden State and Emission Configuration

- **Latent State Space:** The model is restricted to a small state configuration of exactly  $K = 2$  hidden states ( $\mathcal{Y} = \{1, 2\}$ ).
- **Emission Distribution Type:** The emission probabilities must be modeled using continuous **Gaussian distributions**. Each hidden state  $k \in \{1, 2\}$  is parameterized by a scalar mean  $\mu_k$  and a variance  $\sigma_k^2$ .
- **Local Observation Features:** Since the sequences contain scalar observations, the local feature mapping is identity-mapped as  $\phi(x_t) = x_t$  (dimension  $d = 1$ ).

### 6.5.2 Joint Feature Vector Dimensionality

As defined in Section 5.2, your joint feature map  $\Psi(x, z)$  concatenates normalized transition counts and per-state mean emissions. For this course setup ( $K = 2, d = 1$ ), the joint feature vector reduces to a highly compact 6-dimensional representation:

$$\Psi(x, z) \in \mathbb{R}^{2^2 + (2 \times 1)} = \mathbb{R}^6$$

This low-dimensional feature vector is clean, numerical underflow-resistant, and can be efficiently parsed by the downstream scikit-learn One-Class SVM baseline during the retraining iterations.

#### Student Deliverable

Students must submit code, a short report, and the required plots. Their report should compare where OC-SVM fails or succeeds relative to HMAD.

## 7 Conclusion

In this project, we studied anomaly detection from two complementary perspectives. We first introduced the standard one-class view, where the goal is to learn a region that contains normal data and to flag observations that fall outside this region. The OC-SVM baseline provided a classical and practical way to implement this idea using fixed-length feature vectors and kernel methods.

We then saw why this fixed-vector view can be insufficient for sequential data. In many real applications, an observation may look normal in isolation, while its position, ordering, or transition pattern may be abnormal. This motivated the use of Markov chains and hidden Markov models as tools for representing latent sequential structure. In particular, Viterbi decoding allowed us to infer a likely hidden state path for an observed sequence.

The simplified HMAD-style method combined these two ideas. Instead of applying OC-SVM directly to raw or summarized sequence features, we constructed a joint feature representation  $\Psi(x, z)$  that depends on both the observed sequence  $x$  and an inferred hidden state path  $z$ . This representation captures transition behavior and state-dependent emissions, making it possible to detect anomalies that are expressed through hidden temporal structure.

The main lesson of the project is that model assumptions matter. OC-SVM is a strong and useful baseline when the data can be represented well as independent fixed-length vectors. However, when the important information lies in the evolution of a sequence, a sequence-aware representation can be more appropriate. By comparing OC-SVM and the simplified HMAD-style model experimentally,

you should be able to reason not only about which method performs better, but also about why it performs better under a given data-generating process.

A successful project report should therefore go beyond reporting numerical scores. It should explain what kinds of anomalies were present in the data, which features or hidden states helped detect them, where the OC-SVM baseline succeeded or failed, and whether modeling latent sequential structure was actually useful for the chosen experimental setting.